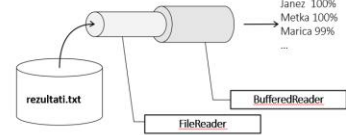


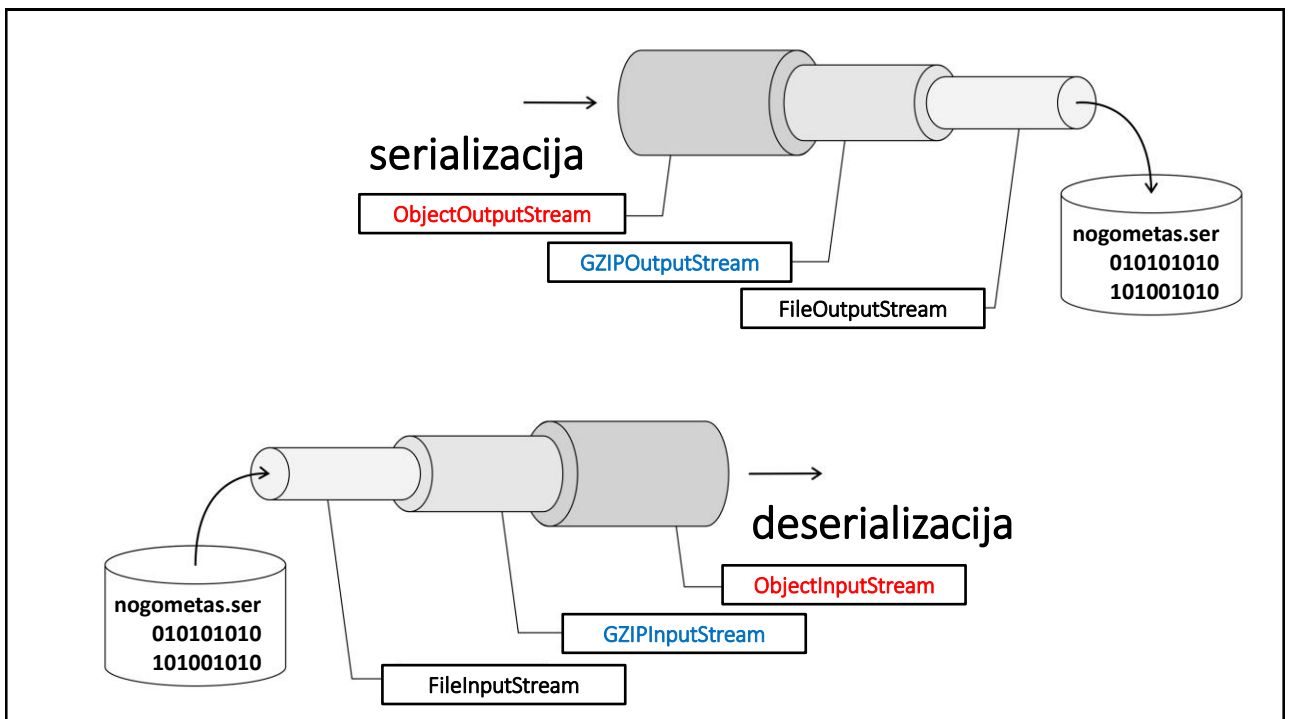
Prejšnjič

- Map, HashMap, TreeSet
- IO tokovi
 - binarni
 - Znakovni
- Serializacija
- Dekorator/Ovojnica



```
FileReader fr = new FileReader("rezultati.txt");
BufferedReader vhod = new BufferedReader(fr);
String vrstica;
while ((vrstica = vhod.readLine()) != null)
    System.out.println(vrstica);
```

1



2

Niti

sočasno izvajanje aktivnosti

uporaba več niti v enem programu za izvedbo različnih opravil

dostop do istih virov

3

Niti (Threads)

- običajno imamo opravka s sekvenčnimi programi - ti **imajo svoj začetek, zaporedje izvajanja in zaključek in v vsakem trenutku le eno točko izvajanja**
- tako je tudi z nitjo, le da ta ni program, saj **sama zase ne more obstajati** oz. teči, izvaja se lahko le v sklopu programa
- izziv: uporaba več niti v enem programu za izvedbo različnih opravil

4

Kaj je nit?

- *Ločeno „izvajanje/procesiranje“, ki teče znotraj enega programa in lahko izvaja neko računsko opravilo neodvisno in sočasno z drugimi nitmi*
- Mnoge aplikacije opravijo svoje delo v sklopu ene same niti: tiste, ki je prožila/klicala `main()` ob zagonu
 - Lahko pa obstajajo dodatne niti...
 - ... Garbage collection teče v niti v ozadju
 - GUI imajo ločeno nit, ki posluša in upravlja dogodke

5

Kaj je nit?

- Nit je vrsta objekta, ki izvaja „neodvisno procesiranje“
 - Potrebno jo je **kreirati kot vsak drug objekt**
 - Nato **zagnati s `start()`**. Posledično se proži določena metoda (podobno kot `main()`). Teče sočasno z drugimi nitmi v istem programu in vidi/lahko dostopa do istih globalnih podatkov
- Dejansko izvajanje se lahko vrši na različnih CPU jedrih, a ni nujno
 - Simulacija niti preko *multipleksiranja* manjšega števila jeder nad večjim številom niti

6

Razred Thread v Javi

- Niti so primerki/objekti razreda **Thread**
 - Lahko jih kreiramo poljubno mnogo, porabljajo prostor in čas
- JVM kreira nit, ki izvaja metodo *main*.
- Niti imajo prioriteto
 - Tiste z višjo prioriteto se preferenčno izvajajo
 - Novo kreirana nit ima privzeto prioriteto identično niti, znotraj katere je bila kreirana (lahko pa prioriteto spremenimo)

7

Razred Thread

- **java.lang.Thread** ograjuje podatke in metode, potrebne za kreiranje samostojnega kontrolnega toka, ki se izvaja na Java interpreterju
- vmesnik **java.lang.Runnable** deklarira operacijo *run()*, ki določa glavni namen oz. nalogo, ki jo izvaja nit
- **java.lang.Thread** implementira vmesnik **java.lang.Runnable**
- da kreiramo nit, lahko oblikujemo podrazred razreda **Thread** in predefiniramo operacijo **run ()**

8

Začetek izvajanja niti

prožimo **start()**

ne `run()` !!!

9

S pomočjo dedovanja

```
class MojaNit extends Thread {

    MojaNit (String ime) {
        setName(ime); // ali super (ime);
    }
    public void run () { ... }
}

new MojaNit("nit1").start();
new MojaNit("nit2").start();
```

Uporabno:

```
Thread.currentThread().getName();
ThreadGroup
```

10

```

public class Student extends Thread {

    private String ime;
    private int stPiv;
    private int casPitja;

    public Student(String ime, int stPiv, int casPitja) {
        this.ime = ime;
        this.stPiv = stPiv;
        this.casPitja = casPitja;
    }

    @Override
    public void run() {

        for (int i=1; i<=stPiv; i++) {
            System.out.println(ime + " pije " + i + ". pivo");
        }
    }
}

```

11

ali pa:

- razred implementira vmesnik **Runnable** in nato kreiramo ciljni objekt, ki ga posredujemo kot argument v konstruktorju niti
- da zagotovimo enkratni nitni objekt lahko uporabimo kar anonimni notranji razred ("anonymous inner class")
- če nek razred, v katerem so na voljo vsi potrebni podatki in metode za izvedbo naloge, že ima nadrazred, ne more istočasno biti tudi naslednik oz. porazred razreda Thread, kar pomeni, da je implementacija vmesnika **Runnable** edina možna rešitev

12

Z implementacijo vmesnika

```
public class Izvedljivi implements Runnable {
    public void run () {}
}

new Thread(new Izvedljivi(), "nit1").start();
```

13

```
class Abstinent implements Runnable {

    @Override
    public void run() {
        for (int i=1; i<3; i++) {
            System.out.println(" Abstinent pije "+ i + ".vodo");
        }
    }
}

Abstinent cudak = new Abstinent();
Thread nit = new Thread(cudak);
nit.start();
```

14

```

12*      Thread x = new Thread ("Anonimni") {
13*          @Override
14          public void run() {
15              for (int i=1; i<100; i++) {
16                  System.out.println(" Anonimni alkoholik ga šele pije");
17                  try {
18                      sleep(500);
19                  } catch (InterruptedException e) {
20                      // TODO Auto-generated catch block
21                      e.printStackTrace();
22                  }
23              }
24          }
25      };
26      x.setDaemon(true);
27      x.start();

```

15

Smrt

- **Dead**

nit lahko zamre na dva načina - po "naravni" poti (metoda *run* se normalno zaključi) ali ker jo "ubijemo" oz. stopiramo s pomočjo metode *stop()* - ta "deprecated" metoda proži oz. pošlje **ThreadDead** izjemo, zato asinhrono "umiranje" (ko pač prejme izjemo)

- **isAlive()** metodo lahko uporabimo za ugotavljanje stanja (vrne true, če ni dead ali new thread)
- izjema **IllegalThreadStateException** nastopi, ko prožimo metodo, ki v aktualnem stanju ni dovoljena, možna

16

Prioritete niti

- prioriteta niti pove Javinemu razvrščevalniku kdaj naj se nit izvaja glede na druge niti
uporablja se deterministični algoritem razvrščanja (fixed priority scheduling)
- nastavimo s **setPriority(int)**

```
MIN_PRIORITY = 1; MAX_PRIORITY = 10;
NORM_PRIORITY = 5;
```
- za izvajanje se izbere nit z najvišjo prioriteto, prihod niti z večjo prioriteto prekine izvajanje tiste z manjšo
- če prioritete ne določimo, se prevzame prioriteta niti, znotraj katere je bila kreirana

17

Pazimo - brez predpostavk

- niti nudijo iluzijo sočasnega izvajanja tudi na enoprocesorskih sistemih - potreben razvrščevalnik (thread scheduler),
- “problem” sebičnih niti
- Java izvajalno okolje ne implementira in garantira dodeljevanje časovnih rezin (time-slicing) - odvisno od sistema, na katerem teče
- torej se ne smemo zanašati, uporabimo pa lahko **yield()** - umik, ki omogoči izvajanje drugim nitim z isto prioriteto, ne pa tudi nižjo

18

Shema menjave niti

- Izpraznitvena shema (preemptive scheduling) – nit z višjo prioriteto vedno prevzame procesorski čas niti z nižjo prioriteto v trenutku, ko postane izvedljiva
- Vedno pa se ne izvaja nit z največjo prioriteto, saj dodeljevalni algoritem prepreči stradanje niti (starvation)
- V operacijskih sistemih, ki podpirajo časovne rezine (“time slicing”), se niti procesor odvzame po preteku določenega časa, če je nit iste prioritete, jo nadomesti – nit iste prioritete po principu “round robin”

19

Daemon niti

- prožimo **setDaemon(true)**
- “daemon” niti zagotavljajo storitve drugim nitim, vsaka Java nit je lahko “daemon” nit
- metoda **run** demonske niti je običajno neskončna zanka, ki čaka na zahteve po storitvah
- ko je edina preostala nit v procesu demonska nit, se izvajanje zaključi (izhod iz interpreterja), saj ni več uporabnikov storitev
- **isDaemon()** pomaga ugotoviti, ali je nit demonska

20

Skupine niti (Thread Group)

- vsaka nit je član skupine niti (Thread Group)
- razred ThreadGroup zagotavlja mehanizme za grupiranje niti in skupno manipulacijo nad nitmi, saj definira in implementira zmožnosti skupine povezanih niti
- Npr. določitev prioritete

21

Določitev skupine

- pri konstruktorju - če nič, pač gre v privzeto skupino - v isto skupino, v kateri je nit, ki je kreirala to novo
- premeščanje med skupinami ni možno

```
ThreadGroup mojaSkupina = new ThreadGroup("NitiOPJ");
Thread mojaNit = new Thread (mojaSkupina,"nit za mojo skupino");

mojaNit.getThreadGroup();
```

- začne se s skupino main skupine lahko vključujejo druge skupine na vrhu je main

22

Metode skupine niti

- `activeCount()`
- `enumerate(polje)`
- `getMaxPriority()`, `setMaxPriority()`
- `getDaemon()`, `setDaemon`
- `getName()`
- `getParent`, `parentOf()`
- *deprecated metode:*
`resume()`, `stop()`, `suspend()`

23

Možni problemi

- problem stradanja (neka nit ne pride do vira in ne more napredovati)
- smrtni objem (deadlock) - poseben primer, ko čakata dve ali več niti in sicer na pogoj, ki nikakor ne more biti izpolnjen - najpogosteje niti čakajo ena drugo
- volatile atributi - spremenljivke objektov

24

Težave pri deljenih virih

```
class Stevec {
    private int c;

    public void increment() {
        c++;
    }
    public void decrement () {
        c--;
    }
}
```

1. Pridobi vrednost c
2. Povečaj c
3. Shrani c

Nit A: Pridobi vrednost c

Nit B: Pridobi vrednost c

Nit A: Povečaj c

Nit B: zmanjšaj C

Nit A: Shrani c

Nit B: Shrani c

25

Race Conditions



- “race condition” se pojavi, ko dve ali več niti sočasno dostopa do iste spremenljivke ali objekta in vsaj ena izvaja posodobitve
- Primer: n1 in n2 sočasno izvajata stavek $x = x + 1$; nad istim statičnim globalnim x.
 - Interno to pomeni nalaganje x, povečanje za 1, shranjevanje x
 - Če n1 in n2 to počneta sočasno, se izvede stavek dvakrat, a lahko se zgodi, da bo x povečan le enkrat
 - n1 in n2 tekmujeta (“race”) pri izvedbi posodobitve

26

Race Conditions

- Predpostavimo, da je x na začetku 5

Thread n1

- LOAD x
- ADD 1
- STORE x

Thread n2

- ...
- LOAD x
- ADD 1
- STORE x

- ... po zaključku $x=6$! **Izgubimo eno posodobitev!**

27

Niti

Thread, vmesnik Runnable

zagon s `start()`

- niti in sinhronizacija pri dostopu do deljenih virov
`synchronized`, `wait`, `notify`

```
class Stevec {
    int vrednost;
    void povecaj() {
        synchronized (this) {
            vrednost++;
        }
    }
}
```

28

Primer – srečni scenarij

```
private Stack<String> stack = new Stack<String>();

public void doSomething() {
    if (stack.isEmpty()) return;
    String s = stack.pop();
    //do something with s...
}
```

Niti A in B želita klicati `doSomething()`,
Na skladu je en element

1. nit A testira `stack.isEmpty()` ⇒ false
2. nit A proži `pop()` ⇒ sklad je sedaj prazen
3. nit B testira `stack.isEmpty()` ⇒ true
4. nit B zgolj return – ne naredi ničesar

29

Primer – „problematični“ scenarij

```
private Stack<String> stack = new Stack<String>();

public void doSomething() {
    if (stack.isEmpty()) return;
    String s = stack.pop();
    //do something with s...
}
```

Niti A in B želita klicati `doSomething()`,
Na skladu je en element

1. nit A testira `stack.isEmpty()` ⇒ false
2. nit B testira `stack.isEmpty()` ⇒ false
3. nit A proži `pop()` ⇒ sklad je sedaj prazen
4. nit B proži `pop()` ⇒ **Exception**

30

Sinhronizacija

- Java zagotavlja mehanizme za preprečitev tovrstnih težav, moramo pa sami eksplicitno (in pazljivo) poskrbeti za to – ni samodejno!
 - “synchronization barrier”
 - Neke vrste zaklepanje
 - Četudi več niti hkrati želi pridobiti *lock*, ga lahko v določenem trenutku uspešno pridobi le ena, ostale morajo počakati
 - Ko se *lock* sprosti, ga lahko pridobi naslednja nit
 - Ne moremo predvideti/napovedati zaporedja, v katerem bodo niti pridobile dostop, če so istih prioritet bo dodeljevanje „pošteno“

31

Rešitev z sinhronizacijo

```
private Stack<String> stack = new Stack<String>();

public void doSomething() {
    synchronized (stack) {
        if (stack.isEmpty()) return;
        String s = stack.pop();
    }
    //do something with s...
}
```

sinhroniziran blok

- Kritične operacije damo v „**synchronized block**“
- objekt **stack** deluje kot „lock“
- Samo ena nit lahko ima „lock“ v določenem trenutku

32

Rešitev – zaklepanje

- Zaklenemo lahko poljubni objekt, tudi **this**

```
public synchronized void doSomething() {  
    ...  
}
```

je ekvivaletno

```
public void doSomething() {  
    synchronized (this) {  
        ...  
    }  
}
```

33

Mrtvi objem

- Slaba stran zaklepanja – mrtvi objem
- Pojavi se, ko niti, ki medsebojno tekmujejo, čakajo na drug drugo... večno!
- Primer:
 - Nit n1 kliče sinhroniziran b znotraj sinhroniziranega a
 - Nit n2 kliče sinhroniziran a znotraj sinhroniziranega b
 - n1 čaka na n2... in n2 čaka na n1...

34

Podrobna sinhronizacija

- Java omogoča natančnejšo sinhronizacijo
 - uporabimo jo lahko le **znotraj** sinhroniziranega bloka
 - klici wait/notify

35

wait/notify

Primer: koda znotraj objekta animator:

```

boolean isRunning = true;

public synchronized void run() {
    while (true) {
        while (isRunning) {
            //do one step of simulation
        }
        try {
            wait();
        } catch (InterruptedException ie) {}
        isRunning = true;
    }
}

public void stopAnimation() {
    animator.isRunning = false;
}

public void restartAnimation() {
    synchronized(animator) {
        animator.notify();
    }
}

```

mora biti sinhronizirana

spusti lock nad animator – jem počaka na notify

obvestilo nitim, ki čakajo na animator „lock“

36

Sinhronizacija

- problem skupnega vira (zbirka, objekt)
primer proizvajalec, potrošnik
pogojne spremenljivke
- uporaba monitorjev - ko ima nit monitor nad določenim objektom, so ostale niti zaklenjene in ne morejo dostopat do/uporabljat nobenega izmed sinhroniziranih delov
- segmenti kode, ki dostopajo do istih podatkov so kritične sekcije, v Javi jih označimo s **synchronized** kar celotno metodo [skupna.java](#)
- dve sinhronizirani metodi istega objekta ne moremo istočasno prožiti!

37

Sinhronizacija

```
class Skupna {
    private int vsebina;
    private boolean available = false;

    public synchronized int get() {
        //monitor-semafor je postavljen
        while (available == false) {
            try {
                wait(); // monitor-semafor se sprosti
            } catch (InterruptedException e){}
        }
        available = false;
        notify(); // monitor-semafor se sprosti
        return vsebina;
    }

    public synchronized void put (int vrednost) {
        //monitor-semafor je postavljen
        while (available == true) {
            try {
                wait(); // monitor-semafor se sprosti
            } catch (InterruptedException e){}
        }
        vsebina = vrednost;
        available=true;
        notify(); // monitor-semafor se sprosti
    }
}
```

38

Metodi **notify** in **wait**

- prožimo jih lahko le v sinhronizirani metodi oz. bloku ali stavku
- **notify()** izbere eno izmed čakajočih niti in jo zbudi
- **notifyAll()** zbudi vse čakajoče niti, ki nato tekmujejo za monitor
- **wait()** - povzroči, da gre aktualna nit na čakanje

39

Razlika

- **sleep(n)** says *"I'm done with my timeslice, and please don't give me another one for at least n milliseconds."* The OS doesn't even try to schedule the sleeping thread until requested time has passed.
- **yield()** says *"I'm done with my timeslice, but I still have work to do."* The OS is free to immediately give the thread another timeslice, or to give some other thread or process the CPU the yielding thread just gave up.
- **wait()** says *"I'm done with my timeslice. Don't give me another timeslice until someone calls notify()."* As with sleep(), the OS won't even try to schedule your task unless someone calls notify() (or one of a few other wakeup scenarios occurs).

40

Monitor

- Vsak objekt ima ključ (**lock**) oz. **monitor**
- V nekem trenutku lahko dostop kontrolira oz. ima le ena nit
- Nit, ki želi prožiti sinhronizirano kodo objekta, mora pridobiti monitor (ključ)
- Če dobi ključ, je OK, sicer mora počakati na ključ
- Šele ko pridobi ključ/monitor, je pripravljena za nadaljevanje

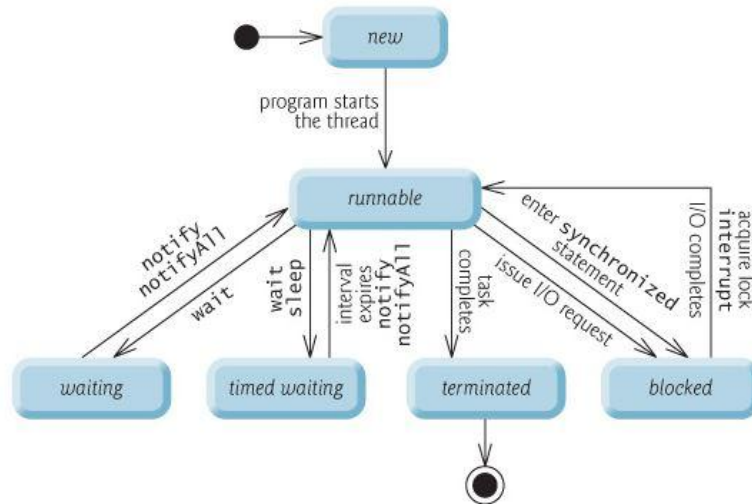
41

Thread.State

- NEW
- RUNNABLE
- BLOCKED
- WAITING
- TIMED_WAITING
- TERMINATED

42

<https://howtodoinjava.com/java/multi-threading/java-thread-life-cycle-and-thread-states/>



43

Sinhroniziran le blok kode

```
synchronized (refObjekta) {
}
```

poljubnega objekta – pazljivo!

```
synchronized (this) {
}
```

Priporočeno: **synchronized** na nivoju metod!

44

Green vs. Native niti

- **green threads** are [threads](#) that are scheduled by a [runtime library](#) or [virtual machine](#) (VM) instead of natively by the underlying [operating system](#).
- **V Java 1.1 še zelene, nato pa opuščene**
- zelene učinkovitejše pri aktivaciji(start) in sinhronizaciji, „native“ pa pri večjedrnih procesorjih, IO, preklopu konteksta itd.

https://en.wikipedia.org/wiki/Green_threads